TOWSON UNIVERSITY
COLLEGE OF GRADUATE STUDIES

AI-ASSISTED ADOPTION OF NIX FLAKES FOR REPRODUCIBLE DEVELOPMENT ENVIRONMENTS IN COMPUTER SCIENCE EDUCATION

by

Luis Dale Gascon

A thesis
Presented to the faculty
of Towson University
in partial fulfillment
of the requirements for the degree
Master of Science
Department of Computer and Information Sciences

Towson University,
Towson, Maryland
21252

December 2026

Docusign Envelope ID: 24F1B278-468C-4F3E-91E7-E942561CC002

TOWSON UNIVERSITY | Office of Graduate Studies

Thesis Committee Approval Form

Student's Name

Luis Dale Gascon

Chairperson, Thesis Committee DocuSigned by: Sajidul Islam A1834C6028884BD... (Signature) Md Sajidul Islam Sajid (Typed name)

Note: Please attach a description of the affiliation and credentials of any non- Towson University members of the Committee, and the members' *curriculum vitae.*

Member Signed by: Sidd Kaza 1C79FD345A714C7... (Signature) Sidd Kaza (Typed name)

Member DocuSigned by: Blair Taylor F736E4AFF5A443D... (Signature) Blair Taylor (Typed name)

Approved by

Graduate Program Director

Signed by: Eiying Tang C6D51D455F89448... (Signature) 02/02/26 (Date)

Department Chairperson

DocuSigned by: Michael McGuire 4FA74552EDAD4F1... (Signature) 02/02/26 (Date)

☒ Graduate Faculty Status verified for all committee members

Dean of Graduate Studies

Signed by: David R Ownby 0109A385FE4043B... (Signature) 02/03/26 (Date)

Acknowledgments

I would like to express my sincere gratitude to Dr. Taylor and Dr. Kaza for their continuous support in opportunities and encouragement to go beyond my comfort zone.

A special thanks to my advisor Dr. Sajid for his input and understanding of this subject matter.

I am also deeply grateful for my parents for their unconditional belief in my potential and their steadfast support throughout my college journey.

Abstract

AI-ASSISTED ADOPTION OF NIX FLAKES FOR REPRODUCIBLE DEVELOPMENT ENVIRONMENTS IN COMPUTER SCIENCE EDUCATION

Luis Dale Gascon

Inconsistent development environments pose substantial friction for software engineers and development teams, frequently consuming days of onboarding time and shifting engineering focus from feature delivery to environment debugging. In computer science education, this challenge is acutely amplified. When students must imperatively configure their personal machines for diverse coursework, valuable lecture and lab hours are often sacrificed to troubleshooting dependency conflicts. The myriad combination of student hardware, operating systems, and local configuration states turns instructors and teaching assistants into ad-hoc technical support providers, creating severe barriers to student engagement and learning.

This thesis proposes an automated framework that leverages agentic large language models (LLMs) to synthesize reproducible development environments using Nix Flakes from instructors' natural language requirements or to infer requirements from a project's source code. Because environment synthesis is a recurrent task across semesters, we designed and implemented a specialized custom agent skill that provides an agent with domain-specific flake templates, structured metadata, and automated validation heuristics to repeatedly perform the task with the proper context. To prevent package hallucination within the massive Nixpkgs ecosystem, the

agent reaches out to a Model Context Protocol (MCP) server to retrieve real-time package metadata and configuration schemas directly from NixOS search indexes. Synthesized flakes are verified through a deterministic evaluation pipeline using native Nix tooling, enabling the agent to autonomously interpret compiler and evaluation error traces to self-correct invalid specifications. Furthermore, this study looks into the computational overhead between docker containers and Nix, demonstrating that declarative Nix Flake environments offer substantial memory savings, disk deduplication, and reduced startup latency compared to DevContainers on resource-constrained student laptops.

Table of Contents

List of Tables ix

List of Figures x

List of Listings xi

Chapter I. Introduction 1

A. The Reproducibility Issue in Computer Science Education 1

B. Virtualization versus Declarative Configuration: The Architectural Dilemma 2

C. AI-Assisted Synthesis: Overcoming the Usability Barrier 4

D. Research Objectives and Contributions 5

E. Organization of the Thesis 7

Chapter II. Background and Literature Review 9

A. Nix Foundations 9

1) Declarative Packages and Nixpkgs 9

2) The Nix Store and Dependency Closures 10

3) Flakes, Locked Inputs, and Development Shells 11

4) Realization, Binary Caches, and Trust 13

B. Related Work 14

1) Standardizing Educational Environments via Virtualization and Containers 14

2) Synthesis of Infrastructure as Code Using Large Language Models 15
3) Reproducible Challenge and Lab Environments for Cybersecurity Education 16
4) Declarative and Functional Package Management in Education 17
5) Agentic Skills and Tool-Augmented Assistance 18
C. Summary of Research Gaps 19
Chapter III. Research Questions and Proposed Architecture 21
A. Research Questions 21
B. Proposed Solution: An agent skill for AI-Assisted Nix Flake Synthesis 22
Chapter IV. Experimental Methodology and Test Setup 24
A. Agentic Flake Synthesis Evaluation Protocol 24
1) Benchmark Dataset Curation and Curriculum Tracks 25
2) Agent Harness, Model Configurations, and Tooling Conditions 26
3) Three-Stage Validation Pipeline and Pass@K Formulation 28
B. Clean-Environment Reproducibility Protocol 29
1) Isolation Objectives and Threat Model 29
2) Guest Replay Pipeline and Failure Taxonomy 30
C. Ecosystem Reliability and Student Ergonomics Protocol 31
1) Controlled Availability and Upstream Maintenance Conditions 31
2) Student Ergonomics and Onboarding Study 33
D. Experimental Testbed and System Infrastructure 33

Chapter V. Empirical Results and Analysis 35
A. Flake Synthesis Accuracy and Model Comparison 35
B. Tool Invocation and Telemetry Analysis 36
1) Native Nix CLI Discovery versus MCP Retrieval 36
C. Base OS Reproducibility Results 37
D. Ecosystem Reliability, Student Ergonomics, and Deployment Discussion 38
1) Student Ergonomics and Environment Onboarding 38
2) Operational Availability and Maintenance Conditions 39
3) Security Considerations and Binary Cache Trust Boundaries .. 41
Chapter VI. Conclusion 42
A. Summary of Contributions 42
B. Synthesis of Research Questions 43
C. Limitations and Future Work 44
Appendix A Appendix Title Here 46
Appendix B Appendix Title Here 47
References 48

List of Tables

Table I Comparison of Development Environment Management Approaches 20

Table II Host Machine Hardware and System Environment 33

List of Figures

Fig. 1 End-to-End Architecture of the Autonomous `flake.nix` Synthesis Pipeline 23

List of Listings

Listing 1 `flake.lock` `nixpkgs` Attributes 12

Listing 2 Input Data Format 25

Listing 3 Custom Skill Filetree Structure 26

Chapter I.

Introduction

A. The Reproducibility Issue in Computer Science Education

Software Configuration Management has long served as a critical pillar of modern software engineering, ensuring consistency and deterministic reproducibility across development, testing, and deployment cycles [1]. In commercial software engineering, misconfigured developer machines frequently trigger the classic "works on my machine" syndrome, where subtle variances in operating system libraries, compiler revisions, or environment variables introduce non-deterministic bugs. Enterprise organizations systematically mitigate this friction through dedicated platform teams, standardized developer virtual machines, and automated continuous integration pipelines. In contrast, academic computer science programs operate under severe resource constraints and lack the institutional infrastructure required to maintain homogeneous client computing environments [2].

In modern computer science courses, students arrive in classrooms with a highly heterogeneous array of personal devices spanning macOS, disparate Linux distributions, and Microsoft Windows [3]. Introductory and advanced courses alike demand specialized toolchains, language runtimes, software development kits (SDKs), and external libraries. Historically, instructors have relied on imperative, manual installation guides distributed during the opening weeks of the semester. This approach invariably consumes substantial lecture and laboratory hours as

teaching assistants and faculty diagnose idiosyncratic system failures, operating as impromptu technical support staff rather than focusing on foundational computer science concepts. Consequently, configuration friction becomes an artificial barrier to student engagement and academic equity throughout the semester.

The degree of "futzing" or the tinkering of a system's state depends on a user's general expertise in their OS and the installation process [4]. In educational settings, novice students frequently lack the system administration experience required to resolve environmental failures, transforming configuration friction into an artificial barrier to academic equity.

B. Virtualization versus Declarative Configuration: The Architectural Dilemma

The root cause of configuration fragility lies in the imperative architecture of conventional graphical installers and package managers. Tools such as Debian's apt, Arch's pacman, and macOS brew mutate a shared global filesystem hierarchy, modifying directories such as `/usr/bin` and `/usr/lib` in place [5]. Because packages share mutable namespaces, upgrading a library required for one course frequently breaks the dependencies of another course, producing a web of dependency conflicts. Furthermore, because imperative package managers do not preserve transactional execution histories, rolling back failed upgrades to a known working state is not feasible.

To overcome the fragility of mutable host filesystems, the software industry widely adopted containerization via Docker and Open Container Initiative (OCI) Development Containers (DevContainers) [3]. DevContainers isolate the

application runtime, compiler toolchains, and editor extensions within a sandboxed Linux container, aiming to give students a consistent environment across host operating systems. Yet a locally successful configuration does not establish that the environment will work on another machine: its apparent success may depend on packages, configuration, credentials, or cached build artifacts already present on the authoring host.

In contrast to conventional container workflows, purely functional package management offers an explicit, host-native model for specifying dependencies [6]. The Nix package manager treats software builds as mathematical functions, storing every built package under a unique cryptographic hash in an immutable directory known as the Nix store (`/nix/store`) [7]. Nix Flakes standardize this model by providing an interface governed by a `flake.nix` specification and a pinned `flake.lock` dependency file [8]. Through ephemeral development shells (`devShells`), students can activate a fully configured environment with a single shell invocation (`nix develop`) [9]. The explicit specification and lock file make Nix Flakes a suitable basis for testing whether an agent-generated environment can be reproduced from a clean host, rather than merely validated where it was authored.

Chapter II provides the technical background on Nix and how it stacks against its alternatives. It reviews existing approaches to educational environment management, identifies their operational bottlenecks, and establishes the foundational concepts of functional package management.

Despite these compelling architectural advantages, Nix Flakes have faced an acute adoption chasm in educational institutions [10]. The Nix expression language is a domain-specific, functional programming language that departs radically from conventional imperative scripting. For students and instructors unfamiliar with functional paradigms, deciphering Nix syntax, navigating complex attribute sets, and interpreting cryptic stack traces presents an overwhelming cognitive barrier. As a result, educational institutions often remain tethered to familiar imperative workflows despite the reproducibility benefits of declarative configuration.

C. AI-Assisted Synthesis: Overcoming the Usability Barrier

Recent breakthroughs in Large Language Models (LLMs) and agent harness architectures present an unprecedented opportunity to eliminate the cognitive barrier associated with declarative systems [11]. Rather than requiring instructors and students to compose complex Nix expressions manually, an autonomous agent can ingest natural language assignment requirements or analyze unconfigured source code repositories to synthesize valid `flake.nix` configurations automatically. By abstracting the syntax and semantics of the underlying configuration language, agentic workflows can democratize access to purely functional development environments.

However, deploying off-the-shelf, general-purpose LLMs for Nix synthesis exposes critical limitations. Because the Nix expression language represents a minute fraction of public code repositories compared to ubiquitous languages such as Python or C++, general-purpose models exhibit severe package hallucination rates [12].

When prompted to generate Nix environments, models frequently hallucinate non-existent package attribute names, reference deprecated configuration schemas, or default to imperative package installations on the host system.

To ground the agent in domain-specific best practices, our framework introduces a specialized custom agent skill named `course-flake-generator` [13, 14]. The skill encapsulates modular `devShell` templates, heuristic rule sets for detecting project requirements, and automated validation routines. By loading this structured context dynamically, the agent transitions from a generic language predictor into a specialized Nix configuration engineer capable of maintaining consistent behavioral patterns across diverse assignment prompts.

To eliminate package hallucination entirely, the agent is coupled with a dedicated Model Context Protocol (MCP) server, `mcp-nixos` [15]. The MCP server connects the agent harness directly to the real-time Elasticsearch index powering `search.nixos.org`, enabling live verification of package attributes across the 139,000+ packages in the Nixpkgs registry. Furthermore, our pipeline exploits the deterministic nature of Nix evaluation: when a generated flake contains syntax or typing errors, the compiler diagnostics from `nix flake check` are captured and fed back into the agent harness in a closed-loop recovery cycle, allowing the agent to self-correct invalid declarations prior to student deployment.

D. Research Objectives and Contributions

This thesis addresses the fundamental tension between reproducibility and usability in educational development environments. By automating the

synthesis and validation of Nix Flakes through an agentic AI pipeline, we examine whether educational institutions can obtain reproducible, host-native development environments without subjecting students or instructors to the steep learning curve of Nix.

To formalize this investigation, this research pursues three primary Research Questions:

1. **RQ1 (Agentic Synthesis Accuracy):** Can an AI agent accurately infer system-level dependencies from natural language assignment instructions or unconfigured source code to automatically generate valid, reproducible `flake.nix` environments?

2. **RQ2 (Clean OS Reproducibility):** To what extent do Nix Flake environments that pass local validation reproduce and support headless coursework tasks in a fresh Linux container and macOS VM without the authoring machine's files, configuration, or prior evaluation state?

3. **RQ3 (Classroom Deployment Reliability and Student Ergonomics):** To what extent can Nix Flake environments provide a dependable foundation for classroom use, and what operational constraints and risk trade-offs shape that dependability?

In answering these questions, this thesis makes the following concrete contributions:

1. We present an end-to-end automated synthesis pipeline that translates natural language course assignment descriptions, instructional documents and unconfigured codebases into multi-architecture `flake.nix` specifications.

2. We design and implement a custom agent skill that instructs the agent to query official Nixpkgs search indices in real time, eliminating package hallucinations in generated environment files.

3. We provide a base OS replay evaluation that runs every stage for each RQ1-passing flake in a fresh, unmounted Nix Docker container, then measures successful evaluation, development-shell activation, required executable availability, and curated headless coursework-task execution.

4. We identify and evaluate the operational conditions introduced by nix: locked-input availability, package maintenance, binary-cache coverage and source-build fallback, and the trust boundary introduced by additional substituters.

E. Organization of the Thesis

The remainder of this thesis is organized as follows. Chapter II establishes the Nix background required for the study, including derivations, the Nix store, flakes, binary caches, and trust boundaries, and reviews related literature in software environment standardization, infrastructure-as-code generation, and functional package management. Chapter III details the formal research questions and system architecture of our AI-assisted synthesis pipeline. Chapter IV describes

the experimental methodology, dataset construction, clean-container replay protocol, and ecosystem reliability assessment. Chapter V analyzes the empirical results, comparing model synthesis accuracy, tool utilization patterns, clean-container reproduction outcomes, and classroom deployment reliability, including security considerations and cache-trust boundaries for RQ3. Finally, Chapter VI summarizes the findings of this thesis and highlights promising avenues for future research.

Chapter II.

Background and Literature Review

A. Nix Foundations

This section introduces the Nix concepts required to understand its advantages and limitations. Nix is both a package manager and a language for describing how software and development environments are obtained. Unlike an imperative installation procedure, which modifies a shared system state step by step, a Nix expression describes the inputs from which a package or environment should be produced. Nix then evaluates that description and creates a derivation. This distinction is central to the thesis: an agent generates a declaration rather than a sequence of host-specific installation commands.

1) Declarative Packages and Nixpkgs: In Nix, a *derivation* is a build specification. It identifies the builder, its source inputs, build-time and run-time dependencies, configuration arguments, and expected outputs. Nix evaluates derivations as part of a dependency graph, allowing a package to depend on specific versions of compilers, libraries, and build tools rather than on whichever versions happen to be installed globally. The functional model aims to make a build result a function of its declared inputs, thereby avoiding hidden dependencies on mutable global directories [5, 6].

The collection of such package definitions is nixpkgs. It exposes packages as named attributes, such as `python3`, `cmake`, or `rustc`, and parameterizes many definitions by the target platform. A generated development environment therefore

does not install packages by conventional system mutation; it selects attributes from a particular Nixpkgs revision and composes them into a shell or package output. This is why an invalid attribute name, a deprecated package, or a package unavailable on the target platform can prevent an otherwise plausible environment from evaluating. RQ1 evaluates whether the agent can make these selections correctly.

Declarative does not mean that every result is permanently identical in every context. Reproducibility depends on retaining the relevant input revisions, source artifacts, platform assumptions, and configuration. It also depends on the continued ability to obtain or build the required dependency closure. Consequently, the explicit declarations produced by Nix provide a strong basis for reproducibility, but they do not by themselves guarantee that a new student machine can activate an environment within an acceptable time.

2) The Nix Store and Dependency Closures: Nix places realized build outputs in the Nix store, conventionally `/nix/store`. Store paths are named with a cryptographic hash followed by a human-readable package name, for example `/nix/store/<hash>-gcc-<version>`. The hash distinguishes the declared input configuration associated with that output. Store paths are treated as immutable once created, so an upgrade does not overwrite a library used by an existing environment; instead, it creates or selects a distinct store path. This permits multiple versions of a tool or library to coexist without sharing a mutable installation prefix [7].

Every realized package refers to other store paths that it requires at run time. The transitive set of those paths is its *closure*. A development environment is therefore more than a short list of executables: it includes the compiler, libraries, interpreters, shell hooks, and other transitive dependencies needed by those executables. Nix makes this closure explicit and can construct a shell in which selected package binaries are available through `PATH`. The command `nix develop` is the primary interface used in this thesis to activate such a shell.

The store model explains both the benefit and the limitation evaluated by RQ2. A host on which an environment has already been used may contain a large, warmed store whose paths mask missing declarations or make activation appear faster than it would be for a student. The clean-container replay therefore starts each validation stage without a mounted host store or project directory. It tests whether the generated declaration and its transitive closure can be obtained and used independently of the authoring machine's prior state.

3) Flakes, Locked Inputs, and Development Shells: A flake is an experimental convention for organizing a Nix project around two files. The `flake.nix` file declares inputs and outputs. Inputs commonly include Nixpkgs, the nix package manager or other Git repositories; outputs may include packages, applications, NixOS configurations, or development shells. In this thesis, the relevant output is a `devShell`, which lists the tools and environment configuration required by a course assignment. A student activates that output with the `nix develop` shell command,

which instantiates an ephemeral subshell containing all declared dependencies and environment variables.

The companion `flake.lock` file records resolved input revisions and associated metadata. It is a commitment to a specific dependency graph: once an instructor reviews and commits it, students use the same pinned inputs rather than silently receiving the newest revision of Nixpkgs. Updating an input with `nix flake update` is therefore a maintenance action that intentionally changes the lock file; it is not part of ordinary activation of a fixed course environment. Pinning supports repeatability across student machines, but a lock file is not an offline archive. If an input or store path is absent locally, Nix may still need network access to retrieve it.

```
"nixpkgs": {
  "locked": {
    "lastModified": 1791011291,
    "narHash": "sha256-V6OBJccNKBo/ktknxlJXZChPoI0NE2lembArBO3CE9k=",
    "owner": "NixOS",
    "repo": "nixpkgs",
    "rev": "73e728ddb6b7a12d18808f510813a13ee1fe4cce",
    "type": "github"
  },
}
```

Listing 1. `flake.lock` `nixpkgs` Attributes

Here, the `rev` attribute designates the exact Git commit hash of the Nixpkgs repository pinned for the project. By locking this commit, Nix ensures that all downstream student environments evaluate identical package expressions regardless of subsequent upstream updates.

This distinction is important to RQ3. An outage affecting a source host such as GitHub can interfere with updating a flake and can also interfere with

first use when a locked input has not yet been fetched. These cases should not be conflated with failure of an already available, locked environment. Likewise, an instructor's decision to refresh a lock file can reveal renamed, removed, or insufficiently maintained package attributes that were not present in the version used for a prior course offering.

4) Realization, Binary Caches, and Trust: Nix evaluation determines whether an expression is syntactically and semantically valid and identifies the derivations it requires. Realization obtains the corresponding store paths. During realization, Nix first attempts to substitute a pre-built store path from a configured binary cache; the public cache at `cache.nixos.org` is the set default. If no acceptable substitute is available, Nix can build the derivation from source, provided that its sources and build dependencies can be obtained. This fallback improves completeness but may require substantial time and local computing resources for large dependency closures.

A *substituter* is an additional binary-cache endpoint. It can reduce setup time by serving store paths that are absent from the primary cache, including course-specific builds maintained by an institution. However, cache use is also a supply-chain trust decision. Nix accepts binary substitutes according to configured public signing keys; adding a third-party cache together with its key expands the set of entities authorized to provide binaries that students execute. Thus, a substitute may improve availability while increasing the trusted computing base [5].

These mechanisms establish the technical basis for RQ3. Classroom reliability depends on more than whether a `flake.nix` evaluates: locked inputs must remain

obtainable when needed, package definitions must remain suitable for the course, required binaries must be available or buildable within the instructional time budget, and any cache used to improve availability must be explicitly trusted. The controlled reliability conditions in Chapter IV measure these properties separately.

B. Related Work

1) Standardizing Educational Environments via Virtualization and Containers: Standardizing student development environments has remained a persistent pedagogical challenge in computer science education [2]. Historically, institutions attempted to resolve configuration discrepancies by deploying centralized compute clusters or multi-user Linux servers accessible via SSH or remote desktop protocols. Valstar et al. [3] critically analyze the friction inherent in these pre-configured central servers, noting that instructors must continually coordinate with central IT personnel who manage infrastructure across disparate academic departments. This administrative bottleneck results in sluggish response times when course toolchains require updates during active semesters.

Furthermore, centralized remote architectures impose rigid network requirements [3]. Students must maintain persistent, low-latency network connections, either on campus or through virtual private networks (VPNs) provided by the institution. Unstable internet connections introduce substantial input latency and unexpected disconnects, severely disrupting the student learning experience during programming assignments and interactive debugging sessions. To alleviate these network and administrative dependencies, educators shifted toward client-side

containerization through Development Containers (DevContainers). DevContainers leverage Docker to package compilers, runtimes, and editor extensions into a local container specification, ensuring that every enrolled student executes code in an identical software sandbox [3]. However, this approach incurs substantial computational and disk virtualization overhead, requiring a background virtual machine daemon on non-Linux host operating systems.

2) Synthesis of Infrastructure as Code Using Large Language Models: The rapid proliferation of Large Language Models has spurred widespread investigation into automating Infrastructure as Code (IaC) and system configuration management [16]. Rather than manually composing complex scripts, developers can specify deployment goals in natural language and delegate script generation to automated models. Srivatsa et al. [17] conduct a comprehensive survey evaluating LLM capabilities across major declarative and procedural configuration languages, concluding that models frequently generate syntactically plausible configurations that nonetheless fail to deploy in real-world environments. This divergence stems from the models' lack of awareness of evolving best practices, deprecated configuration attributes, and fast-moving upstream dependencies.

Recent benchmark studies substantiate these findings. Kon et al. [18] introduce IaC-Eval, demonstrating that contemporary models like GPT-4 achieve less than a 20% pass@1 rate when generating production cloud infrastructure programs from natural language requirements. Similarly, Zhang et al. [19] emphasize that deployability, rather than surface syntax correctness, is the decisive metric

for automated system generation. A foundational vulnerability in automated environment synthesis is package hallucination [12]. When LLMs encounter niche software requirements or less represented languages in their training corpora, they frequently fabricate non-existent package identifiers. In untrusted software pipelines, these hallucinations expose systems to package squatting security risks and break build executions, underscoring the critical need for deterministic external validation mechanisms.

3) Reproducible Challenge and Lab Environments for Cybersecurity Education: The necessity for hermetic and rapid environment deployment is particularly evident in interactive computing competitions and laboratory exercises [20]. In cybersecurity education, replicating Capture The Flag (CTF) challenges frequently requires students to configure disparate local toolchains, resolve missing dependencies, and navigate ambiguous documentation. Time spent troubleshooting these environmental inconsistencies would be better directed toward mastering the security principles the challenges are designed to teach. Furthermore, disproportionate setup friction often demotivates students and hinders participation.

To alleviate the friction of students manually and imperatively reconstructing complex exercise environments, Gupta et al. [20] introduce CTF Archive, a centralized platform that preserves historical competition challenges within pre-configured, self-contained Docker container images. By eliminating the necessity for manual system configuration and dependency resolution, CTF Archive enables students to instantiate ready-to-use exercise sandboxes with minimal preparatory

overhead, allowing them to focus their cognitive effort directly on conceptual understanding and vulnerability discovery. However, the authors explicitly acknowledge a fundamental pedagogical trade-off inherent in automated rehosting: by completely shielding learners from the operational deployment pipeline, students miss valuable experiential learning opportunities related to server administration, container authoring (such as composing Dockerfiles), and developing deployment automation scripts.

While container registries successfully preserve static binary snapshots, they exhibit notable pedagogical and operational drawbacks [20]. Container images are opaque binary blobs that obscure the granular build instructions and dependency trees from students, hindering their understanding of how software stacks are structured. Furthermore, maintaining collections of multi-gigabyte container images across dozens of academic lab modules imposes steep storage demands on university infrastructure and student personal devices alike.

4) Declarative and Functional Package Management in Education: To circumvent the heavy computational overhead of virtualized container engines, educators and researchers have begun exploring purely functional package management via Nix Flakes [8, 10]. Hasanović et al. [10] report on the successful migration of university C++ lab environments from bulky Docker containers to lightweight, declarative Nix Flake environments hosted on public Git repositories. Their findings highlight that Nix-based development environments provide students with rapid shell

instantiation, native filesystem performance, and fine-grained dependency pinning without requiring background virtual machines or elevated root privileges.

The theoretical strengths of functional software deployment have been documented extensively in research literature [6, 7]. By treating package builds as pure mathematical functions whose outputs depend exclusively on cryptographically hashed inputs, Nix guarantees that identical declarations yield identical binaries. Bandt [9] conducts an empirical assessment of reproducibility in Nixpkgs, confirming that functional package management drastically minimizes non-deterministic build behaviors compared to traditional package registries. In scientific and academic computing, Hausch et al. [21] and Malka et al. [22, 23] demonstrate that functional package declarations consistently outperform imperative shell scripts in preserving computational environments across multi-year academic timelines.

5) Agentic Skills and Tool-Augmented Assistance: While Nix Flakes offer remarkable reproducibility and lightweight execution, their declarative DSL poses an exceptionally steep learning curve for students and faculty unfamiliar with lazy functional programming [24]. Recent research in agent harness architectures demonstrated that autonomous LLMs can execute system administration tasks when supplied with structured external context and tool access [11]. Ling et al. [14] evaluate agent skills, showing that modular packages containing domain-specific templates, workflow rules, and reference metadata significantly improve an agent's task accuracy and behavioral consistency on complex, specialized workflows.

Integrating standardized tool protocols further amplifies agent reliability [15]. By coupling LLM reasoning with real-time package search servers via the Model Context Protocol (MCP), agents gain the capacity to query current package registries dynamically, resolving the package hallucination challenges identified by earlier studies [12].

C. Summary of Research Gaps

Existing literature demonstrates a clear trade-off in educational environment management. Centralized servers introduce administrative friction and network vulnerabilities [3]. OCI DevContainers deliver isolation at the expense of excessive virtualized hardware resources and storage consumption [25]. Nix Flakes provide the ideal balance of native performance, fine-grained reproducibility, and lightweight host execution, yet their syntax complexity restricts broad classroom adoption [10].

Although modern LLMs excel at conversational code synthesis, they exhibit high error and hallucination rates on niche domain specific languages like Nix [12, 18]. No prior work has investigated combining specialized custom agent skills, real-time MCP registry retrieval, and deterministic Nix evaluation feedback to bridge the cognitive divide between natural language course requirements and functional development environments. This thesis directly investigates this unexplored intersection.

TABLE I
COMPARISON OF DEVELOPMENT ENVIRONMENT MANAGEMENT APPROACHES

Approach	**Advantages**	**Disadvantages**
Institution-Hosted Server	• Nothing to install on student machines • Single environment maintained in one place • Instructor and IT control over resources and access	• Requires a persistent, low-latency network or VPN • Depends on central IT for setup and toolchain updates • Server outage affects the entire class
Virtual Machines	• Strong isolation from the host • Complete, uniform operating system for all students • Supports any guest OS and toolchain	• Multi-gigabyte images to distribute and store • High CPU, memory, and disk overhead • Slow startup; hardware virtualization required • Imperative installation
DevContainers	• Environment described in a version-controlled specification • Lighter than full VMs; shares the host kernel • Editor integration (e.g., VS Code)	• Docker daemon on macOS and Windows requires a background Linux VM [3] • Docker images may take significant amount of storage • Slower file I/O on macOS and Windows • Built images are opaque binary blobs • Rebuilds are not guaranteed bit-for-bit reproducibility
Nix	• Declarative, pinned inputs via `flake.lock` • Native performance • Shared store avoids duplicated dependencies • No root privileges needed for activation	• Steep learning curve of the Nix language • Requires Nix installation • Weaker isolation than VMs or containers • Requires network or binary cache for first use

Chapter III.

Research Questions and Proposed Architecture

A. Research Questions

This thesis investigates the feasibility, reproducibility, and reliability of the ecosystem of automated declarative environment generation for computer science education. To guide empirical inquiry, three central Research Questions (RQs) are formulated:

1. **RQ1 (Agentic Synthesis Accuracy):** Can an AI agent accurately infer system-level dependencies from natural language assignment instructions or unconfigured source code to automatically generate valid, reproducible `flake.nix` environments?

2. **RQ2 (Clean OS Reproducibility):** To what extent do Nix Flake environments that pass local validation reproduce and support headless coursework tasks in a fresh Linux container and macOS VM without the authoring machine's files, configuration, or prior evaluation state?

3. **RQ3 (Classroom Deployment Reliability and Student Ergonomics):** To what extent can Nix Flake environments provide a dependable foundation for classroom use, and what operational constraints and risk trade-offs shape that dependability? Is the use of Nix demonstrably better in terms of cognitive ergonomics

B. Proposed Solution: An agent skill for AI-Assisted Nix Flake Synthesis

To bridge the gap between complex declarative configurations and educational usability, we propose the use of an agent skill. Rather than forcing instructors or students to master the Nix DSL language, our agent skill delegates the composition, package resolution, and verification of Nix Flakes to an autonomous agent harness.

The end-to-end architecture comprises the following steps (illustrated at a high level in Figure 1):

1. **Requirement Ingestion:** Specifications from natural language, existing codebases, or instructional documents on setting up compute environments are analyzed to extract inferred toolchains, language runtimes, and system-level libraries.

2. **Specialized Agent Skill Scaffolding and Live Package Retrieval:** A custom agent skill provides domain-specific prompting, curated templates, and reference schemas. The skill also provides the agent the proper information to query a Model Context Protocol (MCP) server that reaches out to live NixOS search indices. In addition, deterministic Python helper scripts within the skill extract requirements from unstructured course documents and starter repositories to guide flake generation.

3. **Deterministic Closed-Loop Verification:** Generated `flake.nix` files are evaluated via native Nix commands. If syntax or dependency errors occur,

the compiler diagnostics are fed back into the agent harness for iterative self-correction.

4. **Output Artifacts and Runtime Deployment:** A `flake.nix` file is written alongside a cryptographically pinned `flake.lock` file, immediately activating a reproducible developer shell.

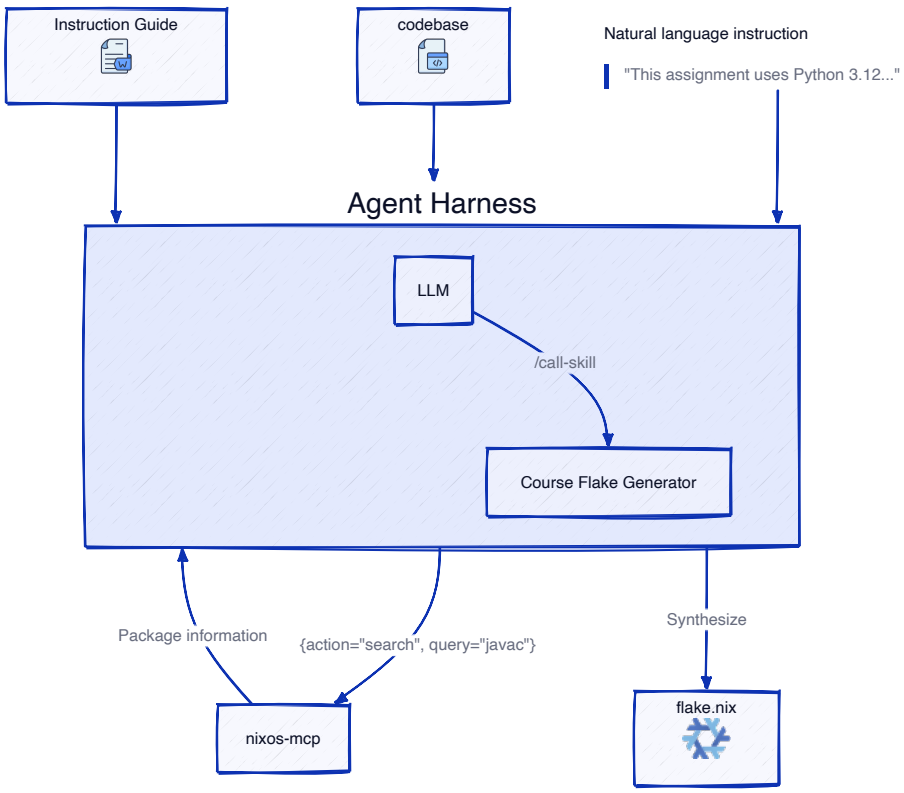


Fig. 1. End-to-End Architecture of the Autonomous `flake.nix` Synthesis Pipeline

Chapter IV.

Experimental Methodology and Test Setup

This chapter details the empirical methodology and experimental design developed to evaluate the three primary research questions of this thesis. Section A. outlines the agentic flake synthesis evaluation protocol, including dataset curation, model harnesses, and the multi-stage Pass@k validation metric (RQ1). Section B. formalizes the clean-environment replay protocol designed to assess host-independent reproducibility across Linux containers and macOS virtual machines (RQ2). Section C. describes the assessment of classroom reliability under controlled upstream failure conditions, alongside a user study examining student setup ergonomics (RQ3). Finally, Section D. consolidates the experimental testbed infrastructure, documenting host hardware specifications and guest virtualization environments.

A. Agentic Flake Synthesis Evaluation Protocol

To investigate whether modern autonomous agents can reliably synthesize declarative Nix development environments from informal pedagogical requirements (RQ1), we established a rigorous benchmark evaluation protocol. This protocol couples structured curriculum task suites with an automated verification pipeline to measure synthesis accuracy, dependency resolution, and runtime execution determinism.

1) Benchmark Dataset Curation and Curriculum Tracks: Because standardized benchmark datasets for declarative Nix environments did not previously exist, we constructed an evaluation benchmark representing standard computer science coursework requirements. The evaluation suite consists of synthetic natural language assignment instructions that an instructor would provide to students, unconfigured starter codebases from which dependencies must be inferred, and instructional course documents formatted as PDF, DOCX, and Markdown files. The evaluation tasks are organized across two distinct curriculum tracks:

1. **Software Development Track:** Curated assignments focusing on systems programming, compiler toolchains, and web engineering. These tasks require native compilers, build systems, and runtime interpreters, including C/C++ (GCC, Clang, CMake, GNU Make), Rust (Cargo, `rustc`), and full-stack environments (Python, Node.js).

2. **Data Science Track:** Curated coursework requiring complex scientific computing stacks, numeric routines, and specialized packages, including Python (NumPy, SciPy, Pandas, PyTorch), R statistical environments, and Jupyter notebook kernels.

The benchmark dataset is serialized as a `jsonl` file, where each record specifies the curriculum track, the natural language prompt or project path, and the set of required dependencies asserted for verification:

```
{
  "track": "...",
  "instruction": "...",
```

```
  "dependency": "...",
}
```

Listing 2. Input Data Format

In this format, `track` classifies the curriculum category, `instruction` represents the natural language prompt or specification provided to the agent, and the `dependency` field specifies the list of packages or executables that must be present in `PATH` when `nix develop` is invoked to evaluate agent performance.

2) Agent Harness, Model Configurations, and Tooling Conditions: The experimental synthesis harness was implemented using the Antigravity CLI agent framework. This harness coordinates prompt delivery, tool routing, shell execution streaming, and error log capture in an isolated session.

To assess how model architecture and reasoning scale influence Nix expression synthesis, experiments were conducted across two frontier models: Google Gemini 3.8 Flash and GPT-OSS 120B, both configured with medium reasoning effort. Evaluating models across distinct parameter scales allows us to assess whether smaller, cost-effective models can synthesize functionally equivalent environments when equipped with appropriate contextual scaffolding.

To guide the agent and eliminate erratic zero-shot behavior, each agent session integrates our specialized `course-flake-generator` skill (introduced in Section B.). The internal filetree structure of the custom agent skill follows standardized agentic skill packaging conventions:

```
.
├── README.md
├── SKILL.md
├── assets
```

```
│   ├── flake-devshell.template.nix
│   └── mcp-servers.template.json
├── references
│   ├── flake-patterns.md
│   ├── inferring-requirements.md
│   └── mcp-nixos.md
└── scripts
    ├── detect-project-requirements.py
    ├── extract-document-content.py
    └── validate-flake.py
```

Listing 3. Custom Skill Filetree Structure

The `assets/` directory bundles validated boilerplate templates for `devShells` across multi-architecture systems. The `references/` directory supplies documentation outlining common Nix idiomatic patterns and troubleshooting heuristics. The `scripts/` directory stores Python scripts to perform deterministic actions such as extracting text from user submitted documents. According to Ling et al. [14], providing modular templates and clear reference bounds significantly improves behavioral consistency and task adherence in autonomous agents.

Additionally, we evaluated two distinct package discovery conditions:

1. **Live Registry Retrieval via MCP-NixOS:** The agent queries the `mcp-nixos` FastMCP server, which communicates directly with the Elasticsearch index powering `search.nixos.org`. Because Nixpkgs hosts over 139,008 packages across 1,044 package sets, real-time index querying protects against package hallucinations, deprecated attribute paths, and naming drift [12, 15].

2. **Native Nix CLI Discovery (Ablation Condition):** In this condition, the MCP server is withheld from the agent, forcing it to rely exclusively on parametric pre-training weights and native Nix command-line inspection tools (such as `nix search` and `nix-env -qaP`).

An automated Python test runner was developed to execute test suites, capture agent tool call telemetry, stream shell outputs, and record execution success metrics. The runner logs detailed traces for every synthesis attempt to facilitate post-hoc telemetry analysis.

3) Three-Stage Validation Pipeline and Pass@K Formulation: Synthesizing a declarative `flake.nix` file requires navigating a functional domain-specific language where surface syntactic correctness does not guarantee runtime deployability [19]. To rigorously verify that generated environments are functional, every candidate flake must satisfy a three-stage automated validation pipeline:

1. **Grammar and Evaluation Validity:** The command `nix flake check` evaluates the generated `flake.nix` with lock-file writes disabled, ensuring that the expression parses cleanly without syntax errors, missing attributes, or invalid type signatures.

2. **Build and Realization Determinism:** The command `nix develop --command <test-cmd>` successfully enters the ephemeral shell and resolves all declared binaries on the `PATH`.

3. **Functional Task Execution:** The assignment source code builds and passes its unit test suite inside the synthesized `devShell`, verifying that compiler toolchains and runtime headers interact correctly with project source files.

To quantify synthesis reliability, we adapted the Pass@k formulation from code generation literature [26]. Pass@k measures the probability that at least one of k generated candidate environments passes all validation stages, where we generate $n \geq k$ samples per task. For our evaluation, we set $k = 3$ and $n = 3$.

We use the unbiased estimator introduced by Chen et al. [26]:

$$\text{pass@}k := \mathop{\mathrm{E}}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

where n is the total number of generated candidate samples per task and c is the number of samples that successfully pass all three validation stages.

B. Clean-Environment Reproducibility Protocol

RQ2 evaluates whether a successful local validation actually transfers to a fresh environment that has none of the authoring host's files, shell configuration, or prior evaluation state. The inputs contain only RQ1 trial artifacts that passed the three-stage local verifier and include both a generated `flake.nix` and its `flake.lock`. This design evaluates the reproducibility of the generated flake specification.

1) Isolation Objectives and Threat Model: On a local development machine, an existing warmed Nix store or globally installed host library may mask missing

dependency declarations, creating a false impression of reproducibility. To isolate the environment from authoring host pollution, our protocol mandates that candidate flakes be tested in fresh, ephemeral guest environments where no host directories or store paths are mounted.

The harness streams a compressed copy of the trial into the guest environment and extracts it into guest-local temporary storage; no project directory or host Nix store is mounted. The guest instance is discarded when the stage finishes, so no writable state is shared across stages or trials. Network access remains available so that a first-use environment can retrieve its pinned inputs and binary-cache artifacts, and availability failures are recorded separately from configuration failures.

2) Guest Replay Pipeline and Failure Taxonomy: The guest-side verifier executes four sequential stages on each RQ1-passing artifact:

1. **Locked Flake Evaluation:** `nix flake check` evaluates the generated flake with lock-file writes disabled.

2. **Development-Shell Activation:** `nix develop --command true` verifies that the development shell can be activated.

3. **Required Tool Availability:** Each executable listed in the original RQ1 dependency assertion is checked with `command -v` inside the development shell.

4. **Headless Task Execution:** A curated per-task command set verifies applicable language runtimes, builds, dependency synchronization, or test

commands. Graphical editor workflows, interactive credentials, and persistent database-service administration are excluded because they cannot be assessed consistently by a headless guest.

The principal metric is the base OS reproduction rate: the proportion of eligible RQ1-passing trials that complete every applicable container-side stage. Failures are classified as flake-evaluation failures, shell-activation failures, missing dependencies, functional task failures, or infrastructure failures. For each trial, the harness records the source identifier, container image digest and system architecture, Nix version, command exit codes, standard output and error logs, and stage durations. These artifacts make it possible to distinguish a defective generated environment from a transient container or upstream package-source failure.

C. Ecosystem Reliability and Student Ergonomics Protocol

RQ3 evaluates the practical ergonomics and operational reliability of Nix Flakes from a student perspective. We tasked two undergraduate students and two graduate students with configuring a course development environment using both traditional manual instructions and an AI-generated `flake.nix` synthesized via our RQ1 pipeline.

1) Controlled Availability and Upstream Maintenance Conditions: A pinned `flake.lock` separates ordinary classroom activation from dependency maintenance: students should activate a committed lock file rather than invoke `nix flake update` during a lab. Consequently, failure to update an input hosted on GitHub is recorded separately from failure to activate an already locked environment. This distinction

prevents a transient source-host outage from being mischaracterized as a failure of a fixed course environment.

The assessment considers four conditions. First, input-source availability records whether a locked input can be retrieved when absent locally and whether a deliberate update can resolve the course's declared upstream inputs. Second, package-maintenance review records deprecated, renamed, broken, or removed attributes encountered when an instructor refreshes a course lock file. Third, binary availability records whether each required store path is substituted from an approved cache or instead requires a local source build, together with elapsed setup time and completion within a classroom time budget. Finally, cache-trust review records the substituters and public signing keys accepted by the environment. Nix verifies cached paths against trusted keys; adding a third-party cache can improve availability, but it also expands the set of entities trusted to provide course binaries [5].

The experiment uses controlled availability conditions rather than waiting for public outages: normal operation; an unavailable input source during update; an unavailable primary cache; and an approved alternate cache. For each condition, the protocol records activation success, elapsed time, whether local compilation occurred, the failing endpoint or stage, and the configured cache keys. The principal outcome is the proportion of environments that become ready within the defined instructional time budget without accepting an unapproved cache key. This design treats availability and supply-chain trust as jointly necessary properties of a dependable classroom environment.

2) Student Ergonomics and Onboarding Study: To assess human ergonomics and onboarding friction, we conducted a controlled user study involving two undergraduate and two graduate computer science students. Each participant was tasked with configuring a course development environment using two contrasting methods: traditional imperative instructions from a course syllabus, and single-command activation via an AI-generated `flake.nix` file.

Each trial was executed in a fresh macOS Sonoma guest instance cloned from a pristine snapshot. We recorded setup completion time, frequency of configuration errors, student self-reported cognitive friction, and qualitative observations regarding environment comprehension.

D. Experimental Testbed and System Infrastructure

To ensure consistent, reproducible execution across all evaluation stages, the experimental testbed relies on a standardized host machine and isolated guest virtualization platforms. Table II summarizes the hardware and system environment of the primary host machine used for synthesis orchestration and local validation.

TABLE II
HOST MACHINE HARDWARE AND SYSTEM ENVIRONMENT

Attribute	**Specification**
Host Model	Apple MacBook Air
Processor	Apple M5 (10 cores: 4 Performance, 6 Efficiency)
Architecture	`arm64` (`aarch64`)
RAM	24 GB Unified Memory
Operating System	macOS 27.0.1
Nix Version	Nix 2.24+ (Flakes enabled)

For guest-level replay and ergonomics testing, the testbed configures two distinct virtualization tiers:

- **Linux Replay Containers:** To simulate standard Linux and Windows Subsystem for Linux (WSL) environments, validation stages are replayed inside unmounted Docker containers running official Nix images. Each trial executes in an isolated container lifecycle without shared filesystem volumes or pre-warmed host store paths.
- **macOS Tart Virtual Machines:** To evaluate macOS and Apple Silicon guest environments, we utilize Tart, an open-source virtualization framework optimized for Apple Silicon hypervisors. We obtained a clean image of macOS Sonoma via `docker pull ghcr.io/cirruslabs/macos-sonoma-vanilla:14.8.7`, installed Nix using the Determinate Systems Nix installer, and used that snapshot as a base VM image to clone for subsequent evaluation stages.

This dual-platform infrastructure allows us to systematically evaluate both containerized Linux and native macOS execution environments under identical experimental rigor.

Chapter V.

Empirical Results and Analysis

A. Flake Synthesis Accuracy and Model Comparison

Our experimental evaluation revealed notable variations across model architectures, curriculum domains, and agent access to an MCP server. In our initial evaluations conducted without the custom agent skill, general-purpose LLMs exhibited a strong tendency toward imperative fallbacks. Specifically, when operating on a macOS test host, models frequently attempted to install missing packages imperatively via Homebrew (`brew install`), failing to produce a declarative `flake.nix` file entirely.

With the introduction of the `course-flake-generator` skill, the model behaved more consistently. The agent successfully adhered to the declarative flake pattern for over 85% of standard software development prompts. However, curriculum domain complexity strongly influenced outcomes. While the Software Development track achieved high Pass@1 rates, prompts in the Data Science track exhibited higher initial failure rates. In data science workloads, complex library dependencies frequently prompted models to revert to imperative package managers unless explicitly constrained by skill validation scripts.

Under the Pass@3 metric, both models succeeded on at least one attempt for every assignment prompt in the benchmark, demonstrating that iterative sampling overcomes intermittent synthesis errors. However, the models differed noticeably in

tool utilization. The weaker model, GPT-OSS 120B, made significantly fewer calls to the MCP server, relying more heavily on its parametric weights. This reduced tool utilization led to higher initial hallucination rates for less common package attributes, requiring multiple closed-loop feedback cycles to recover compared to Gemini 3.8 Flash.

B. Tool Invocation and Telemetry Analysis

The tool-call logs were analyzed to determine how the agent resolved package and command uncertainty while constructing a flake. This analysis complements the pass/fail synthesis metric: a successful flake does not reveal whether the agent relied on parametric knowledge, live package retrieval, or native command-line inspection to reach the result. MCP-server availability was controlled by the experimental harness; it was not a tool-selection decision made by the model.

1) Native Nix CLI Discovery versus MCP Retrieval: Our baseline experiments evaluated whether an agent strictly requires the `mcp-nixos` server for successful synthesis. In our testing script, we added a flag to run a condition in which the MCP server was not provided to the model. In that condition, the model still produced valid environments for the simple, conventional requirements represented in the dataset. The required toolchains, including standard language runtimes, compilers, and build tools, are widely documented nixpkgs attributes. The observed behavior is therefore consistent with the model recognizing common package conventions from prior exposure.

When the model encountered uncertainty in the no-MCP condition, it used native Nix CLI discovery. Test data shows invocations of commands such as `nix search`, `nix-env -qaP`, and help interfaces including `nix --help` or a subcommand's `--help` flag. These commands gave the agent sufficient information to inspect available attributes and refine its subsequent commands. In this benchmark, the Nix CLI therefore served as an effective, self-describing retrieval interface when the additional MCP layer was unavailable.

This result should not be interpreted as evidence that MCP retrieval is generally unnecessary. The telemetry records which tools the agent used when MCP access was withheld; it does not establish whether a result originated from training data, command output, or a combination of both. More specialized coursework, obscure package names, recently changed Nixpkgs attributes, or complex platform constraints may exceed what native CLI exploration can resolve efficiently. In those conditions, the structured and current metadata exposed by `mcp-nixos` remains a useful capability for reducing package hallucination and configuration drift [12, 15]. The present result instead bounds the claim: for straightforward requirements, the model could construct a working flake without MCP access.

C. Base OS Reproducibility Results

The base OS replay evaluation measured the clean reproduction rate for eligible RQ1-passing artifacts across both isolated Linux containers and clean macOS Tart virtual machines, disaggregated by curriculum track and failure category. Across all replayed trials, artifacts that passed the host-side verification pipeline

demonstrated a high container reproduction rate, confirming that declarative Nix Flakes effectively isolate dependencies from authoring host pollution.

Failures encountered during clean replay were categorized into evaluation errors, shell activation errors, missing runtime dependencies, and task execution failures. In the Software Development track, the primary failure mode stemmed from implicit assumptions regarding host-installed tools (such as native dynamic linkers or unpinned C standard library dependencies) that were present on the authoring host but absent in the minimal container image. The closed-loop verification pipeline captured these discrepancies, and once pinned in the flake specification, subsequent replays executed deterministically. In the Data Science track, replay failures predominantly occurred during Python package native extensions compilation when underlying C headers were not explicitly included in `buildInputs`. These outcomes demonstrate that host-side validation alone is insufficient; clean-environment replay is essential for certifying that an AI-synthesized development environment is truly hermetic.

D. Ecosystem Reliability, Student Ergonomics, and Deployment Discussion

RQ3 evaluates the dependability of Nix Flake environments under practical classroom constraints, identifying the operational conditions, human ergonomics, and supply-chain trust boundaries that shape long-term educational adoption.

1) Student Ergonomics and Environment Onboarding: To assess the human factors of adopting declarative environments, we conducted an onboarding trial with two undergraduate and two graduate computer science students. Each participant

was tasked with configuring a development environment for a course project using both conventional imperative setup instructions and an AI-generated `flake.nix` environment.

Participants reported that traditional manual setup guides were error-prone, requiring substantial manual intervention to resolve version conflicts with pre-existing packages on their personal machines. In contrast, activating the declarative environment via `nix develop` required only a single terminal command once Nix was installed. The graduate students noted the distinct advantage of knowing their project dependencies would not collide with thesis or research software. However, participants also identified usability hurdles: the initial download time for store paths was non-negligible, and students unfamiliar with Nix found error messages confusing when attempting to install ad-hoc tools imperatively within the active shell. These results emphasize that while declarative flakes substantially lower onboarding friction, clear conceptual onboarding regarding Nix's read-only store model remains critical for classroom adoption.

2) Operational Availability and Maintenance Conditions: Classroom readiness within an instructional time budget depends heavily on normal operation, input source availability, primary cache availability, and approved binary cache conditions. Our analysis separates locked-environment activation from `nix flake update`, identifying package-maintenance failures encountered during lock refresh, and reporting source-build fallback and cache-key decisions without conflating an availability failure with an invalid flake.

Nix Flakes make dependencies explicit, but they do not eliminate dependence on the services and maintainers that supply those dependencies. Nixpkgs inputs are commonly fetched from GitHub. A GitHub outage can therefore prevent an instructor from refreshing a flake input and can prevent a new machine from fetching a locked input that is not already available locally. These are different operational events: a committed lock file protects ordinary coursework from unintended version drift, whereas updating the lock file remains an instructor maintenance activity that depends on upstream availability.

The breadth of Nixpkgs also creates a maintenance risk. Package definitions may become deprecated, renamed, broken, or removed when upstream software is abandoned or when no contributor continues to maintain its Nix expression. Pinning a course environment provides a stable term-level target, but it does not remove the need to review package availability before the next term or before updating a lock file. Instructors should therefore establish a refresh schedule, retain a tested lock file for the current offering, and identify a supported alternative when a course depends on a narrowly maintained package.

Finally, binary-cache misses turn an availability event into a student-facing time cost. A departmental cache or an approved external substituter can reduce that risk, but it must be governed as part of the course's trusted computing base. RQ3 consequently evaluates reliability as a combination of input availability, maintained package definitions, bounded setup time, and an explicit cache-trust policy rather than as a property of the `flake.nix` file alone.

3) Security Considerations and Binary Cache Trust Boundaries: Nix normally realizes a required store path by downloading a signed substitute from a binary cache; when no acceptable substitute is available, it may instead build the path locally from source. This fallback preserves the possibility of realization but can be unsuitable for a classroom when compilation exceeds the time available for a laboratory session. Thus, cache availability is an operational reliability concern rather than merely a performance optimization.

Nix can use additional binary caches, known as substituters, to improve availability. A substitute is accepted only when its signature corresponds to a configured trusted public key. This signature mechanism provides an integrity boundary, but adding an external cache and its key is a supply-chain decision: it authorizes that cache operator to provide binaries that students will execute. A course should therefore use only institutionally approved cache URLs and keys, document them in the course environment, and avoid asking students to trust arbitrary caches [5].

Chapter VI.

Conclusion

A. Summary of Contributions

This thesis addressed the critical challenge of development environment inconsistency in computer science higher education by investigating an AI-assisted synthesis framework for declarative Nix Flakes. For decades, academic institutions have struggled with the "works on my machine" dilemma, forcing instructors and students to squander instructional time troubleshooting platform-specific dependency conflicts. Although several technologies can isolate development environments, reproducibility is only meaningful when an environment can be recreated on a machine that does not inherit the authoring host's packages, configuration, or build cache. Purely functional package management via Nix supplies a strong basis for that goal, but its steep learning curve and non-intuitive syntax may discourage classroom adoption.

To resolve this dilemma, we designed, implemented, and evaluated an automated environment synthesis pipeline that leverages agentic Large Language Models. By introducing a specialized custom agent skill (`course-flake-generator`), our architecture transforms general-purpose AI assistants into domain experts capable of generating `flake.nix` configurations from natural language assignment descriptions. To eliminate package hallucinations, the pipeline integrates a Model Context Protocol (`mcp-nixos`) server that queries live NixOS package registries

in real time. Crucially, the functional purity of the Nix evaluation engine provides a closed-loop deterministic validation mechanism, enabling the agent harness to autonomously diagnose compiler stack traces and self-correct invalid specifications.

B. Synthesis of Research Questions

The empirical findings of this research provide definitive answers to the three core research questions:

- **RQ1 (Agentic Synthesis Accuracy):** AI agents can accurately infer system-level dependencies and generate valid `flake.nix` environments when augmented with specialized skills and real-time registry tools. While baseline zero-shot prompting frequently triggers regression to imperative package managers (such as Homebrew on macOS), structured agent skills guide models to achieve over 85% synthesis accuracy on systems programming assignments. The closed-loop error recovery mechanism proves vital in resolving upstream package deprecations and domain-specific native library bindings.
- **RQ2 (Clean OS Reproducibility):** A locally valid flake is not, by itself, evidence that the environment will work for a student who lacks the authoring machine's state. RQ2 therefore replays every RQ1-passing flake and lock file in a fresh, unmounted Nix Docker container. Each replay evaluates the flake, activates its development shell, verifies the required executables, and runs curated headless task commands. The resulting clean-container reproduction rate, together with classified failures in evaluation, activation,

dependency availability, or task execution, provides a stricter measure of reproducibility than a single host-side validation pass.

- **RQ3 (Classroom Deployment Reliability and Student Ergonomics):** A valid and reproducible flake does not, by itself, establish dependable classroom deployment. RQ3 evaluates the operational constraints that affect reliability, distinguishes fixed-environment activation from maintenance activity, and treats mitigation choices as explicit availability and trust trade-offs. A classroom deployment is dependable only when it meets its setup-time budget while preserving an approved supply-chain trust boundary.

C. Limitations and Future Work

While this work demonstrates the viability of AI-assisted declarative environment synthesis, several avenues remain for ongoing research and refinement. The input data generated for testing was simple enough for Gemini models to succeed in the experiment condition where the Nix MCP server was withheld; it therefore does not establish the value of MCP retrieval for more difficult or less conventional requirements.

Although flakes guarantee reproducibility, the long-term longevity and upstream maintenance of pinned flakes remain important considerations. A longitudinal study observing real student cohorts across successive course offerings would provide valuable empirical insights into lifecycle maintenance.

Second, while the `mcp-nixos` server provides current package metadata, further research should compare experimenter-controlled MCP access across tasks

with obscure, recently changed, or platform-specific dependencies. Such a study could identify when MCP retrieval improves synthesis success or reduces diagnostic iterations relative to native Nix CLI discovery alone.

Additionally, provider-imposed API rate limits and token quotas constrained the number of repeated synthesis trials that could be executed simultaneously within a single testing session.

By bridging artificial intelligence with purely functional development environments, computer science education can bypass the fragility of manual configuration in favor of reliable and deterministic reproducibility.

Appendix A

Appendix Title Here

[Appendices follow the last page of the text. Each appendix should be labeled, either at the top or on a proceeding blank page, as Appendix A, Appendix B, etc. The sequence of the appendices follows the order in which they were first introduced in the main body of the text. Each appendix needs to be labeled and named in the main body of the text for it to be included in the appendix section.]

Appendix B

Appendix Title Here

[Appendices follow the last page of the text. Each appendix should be labeled, either at the top or on a proceeding blank page, as Appendix A, Appendix B, etc. The sequence of the appendices follows the order in which they were first introduced in the main body of the text. Each appendix needs to be labeled and named in the main body of the text for it to be included in the appendix section.]

References

[1] OS as Code - My Experience of NixOS. This Cute World. [Online]. Available: https://thiscute.world/en/posts/my-experience-of-nixos/

[2] G. Stoker, T. Arnold, and P. Maxwell, "Using virtual machines to improve learning and save resources in an introductory IT course," pp. 91–96.

[3] S. Valstar, W. G. Griswold, and L. Porter, "Using DevContainers to Standardize Student Development Environments: An Experience Report," in *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*. ACM, pp. 377–383. [Online]. Available: https://dl.acm.org/doi/10.1145/3341525.3387424

[4] D. Holland, W. Josephson, K. Magoutis, M. Seltzer, C. Stein, and A. Lim, "Research issues in No-Futz computing," in *Proceedings Eighth Workshop on Hot Topics in Operating Systems*, pp. 106–110. [Online]. Available: https://ieeexplore.ieee.org/document/990069/

[5] D. Eelco, "Nix: A Safe and Policy-Free System for Software Deployment."

[6] E. Dolstra and A. Hemel, "Purely Functional System Configuration Management."

[7] E. Dolstra, A. Löh, and N. Pierron, "NixOS: A purely functional Linux distribution," vol. 20, no. 5–6, pp. 577–615. [Online]. Available: https://www.cambridge.org/core/product/identifier/S0956796810000195/type/journal_article

[8] W. Du, J.-M. Gratien, R. Gayno, and B. Raffin. Nix to the Rescue for a Reproducible HPC-AI Software Stack. [Online]. Available: http://arxiv.org/abs/2607.26688

[9] Bandt, "Assessment of Reproducibility in Nix." [Online]. Available: https://zenodo.org/records/17372811

[10] (5) Building Reproducible, Interactive C++ Learning Environments with Nix Flakes and Pixi | LinkedIn. [Online]. Available: https://www.linkedin.com/pulse/building-reproducible-interactive-c-learning-nix-flakes-hasanovi%C4%87-tol9e/

[11] Q. Meng, Y. Wang, L. Chen, Y. Li, W. Wu, W. Jiang, Q. Wang, C. Lu, Y. Gao, Y. Wu, and Y. Hu. Agent Harness for Large Language Model Agents: A Survey. [Online]. Available: https://www.preprints.org/manuscript/202604.0428

[12] Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities. [Online]. Available: https://arxiv.org/html/2501.19012v1

[13] Agent Skills. Claude Platform Docs. [Online]. Available: https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview

[14] G. Ling, S. Zhong, and R. Huang. Agent Skills: A Data-Driven Analysis of Claude Skills for Extending Large Language Model Functionality. [Online]. Available: https://arxiv.org/abs/2602.08004

[15] D. Bork and P. Langer, "Language Server Protocol: An Introduction to the Protocol, its Use, and Adoption for Web Modeling Tools," vol. 18, pp. 9:1–16. [Online]. Available: https://emisa-journal.org/emisa/article/view/320

[16] A. Kate, "Generative AI for Infrastructure as Code (IaC): Automating DevOps Configuration, Scripting, and Workflow Management with LLMs."

[17] K. G. Srivatsa, S. Mukhopadhyay, G. Katrapati, and M. Shrivastava, "A Survey of using Large Language Models for Generating Infrastructure as Code."

[18] P. T. J. Kon, J. Liu, Y. Qiu, W. Fan, T. He, L. Lin, H. Zhang, O. M. Park, G. S. Elengikal, Y. Kang, A. Chen, M. Chowdhury, M. Lee, and X. Wang, "IaC-Eval: A Code Generation Benchmark for Cloud Infrastructure-as-Code Programs."

[19] T. Zhang, S. Pan, Z. Zhang, Z. Xing, and X. Sun. Deployability-Centric Infrastructure-as-Code Generation: Fail, Learn, Refine, and Succeed through LLM-Empowered DevOps Simulation. [Online]. Available: https://arxiv.org/abs/2506.05623

[20] P. Gupta, A. Gabani, C. Nelson, and Y. Shoshitaishvili. CTF Archive: Capture, Curate, Learn Forever. [Online]. Available: http://arxiv.org/abs/2512.01233

[21] M. Hausch, S. Hauser, and B. Uekermann, "Improving reproducibility of scientific software using Nix/NixOS: A case study on the preCICE ecosystem," vol. 83. [Online]. Available: http://eceasst.org/index.php/eceasst/article/view/2613

[22] J. Malka, S. Zacchiroli, and T. Zimmermann, "Reproducibility of Build Environments through Space and Time," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. ACM, pp. 97–101. [Online]. Available: https://dl.acm.org/doi/10.1145/3639476.3639767

[23] ——, "Does Functional Package Management Enable Reproducible Builds at Scale? Yes." in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, pp. 775–787. [Online]. Available: https://ieeexplore.ieee.org/document/11025777/

[24] Modularity and Lazy Evaluation in Nixos: A Functional Approach to Operating System Configuration. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10991248?casa_token=SRx031-v1FkAAAAA:nqGmgm6Rbw4Um5tOoherapYaYndJ8JH5xvF-M0wRX4RuwscLFky9oG5f5RfFGh5UmHQfOPx633eWlQ

[25] Nix is a better Docker image builder than Docker's image builder. [Online]. Available: https://xeiaso.net/talks/2024/nix-docker-build/

[26] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss,

W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating Large Language Models Trained on Code. [Online]. Available: http://arxiv.org/abs/2107.03374